Chapter 5

遞迴

5.1 前言

- 程式模組化
- 副程式 (或函數 (式)(Function))
- 某一處理程序，在處理的過程中又反覆地執行該處理程序，我們稱這種現象為遞迴 (Recursive) 。
- 如果撰寫程式來實作上述處理程序，將程式包裝成一支獨立的函數，該函數便可以反覆地自己呼叫自己，因此我們稱這個函數為遞迴函數。

5.2 遞迴函數的運作原理

- 計算 2 的 5 次方

$$2^5 = 2 \times 2^4$$
$$= 2 \times (2 \times 2^3)$$
$$= 2 \times 2 \times (2 \times 2^2)$$
$$= 2 \times 2 \times 2 \times (2 \times 2^1)$$

$$2^n = 2 \times 2^{n-1}, \quad 2^{n-1} = 2 \times 2^{n-2}$$

5.2 遞迴函數的運作原理

- 撰寫一個計算 2 的 n 次方的遞迴函數(式)程式，則該函數(式)將具備兩個主要特徵：
  1. 該遞迴函數(式)可以自己反覆地呼叫自己
     - 第一次呼叫時的引數為 n ，
     - 第二次呼叫時的引數為 n-1 ，
     - 第三次呼叫時的引數為 n-2 ，... ，
     - 引數逐次遞減。
  2. 當引數值等於 1 時，必須停止遞迴呼叫。

5.2 遞迴函數的運作原理

- 用下列 C 語言程式來實作

```
long two_power_n(long n)
{
    if(n == 0)
        return 1;
    if(n == 1)
        return 2;
    if(n > 1)
            return (2 * two_power_n(n-
    1));
    else
        printf("\n錯誤！ n 必須為正整數！\n");
}
```

5.2 遞迴函數的運作原理

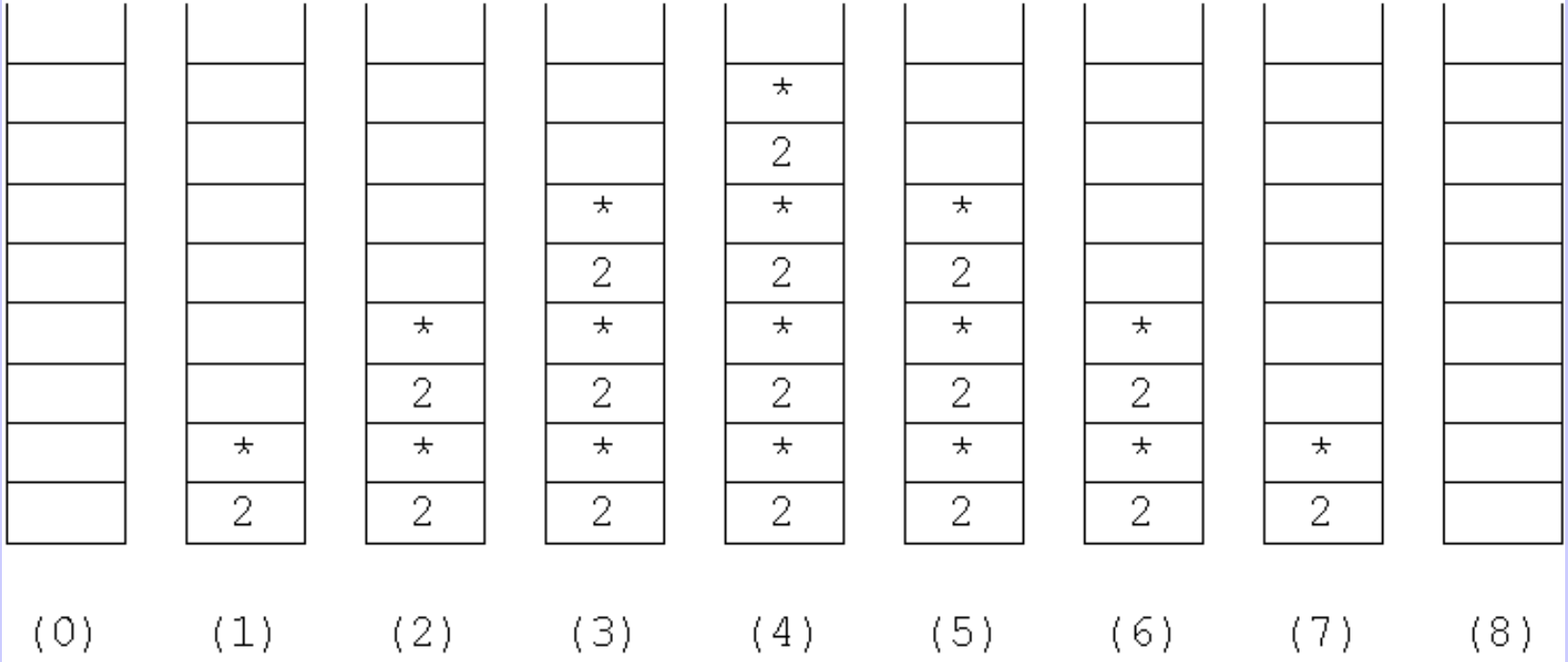


圖 5.1 執行 two_power_n(5) 函數(式)時堆疊的變化情形

5.3 撰寫遞迴函數程式的訣竅

- 撰寫遞迴函數程式的訣竅如下：
  1. 為遞迴函數取一個響亮的名稱？
  2. 思考遞迴函數須要哪些引數？及每一個引數的資料型態？
  3. 遞迴函數自己呼叫自己時要帶入的參數值為何？
  4. 遞迴函數的回傳值為何？如何在回傳時呼叫自己？
  5. 遞迴函數的終止條件為何？終止遞迴時的回傳值為何？

5.3 撰寫遞迴函數程式的訣竅

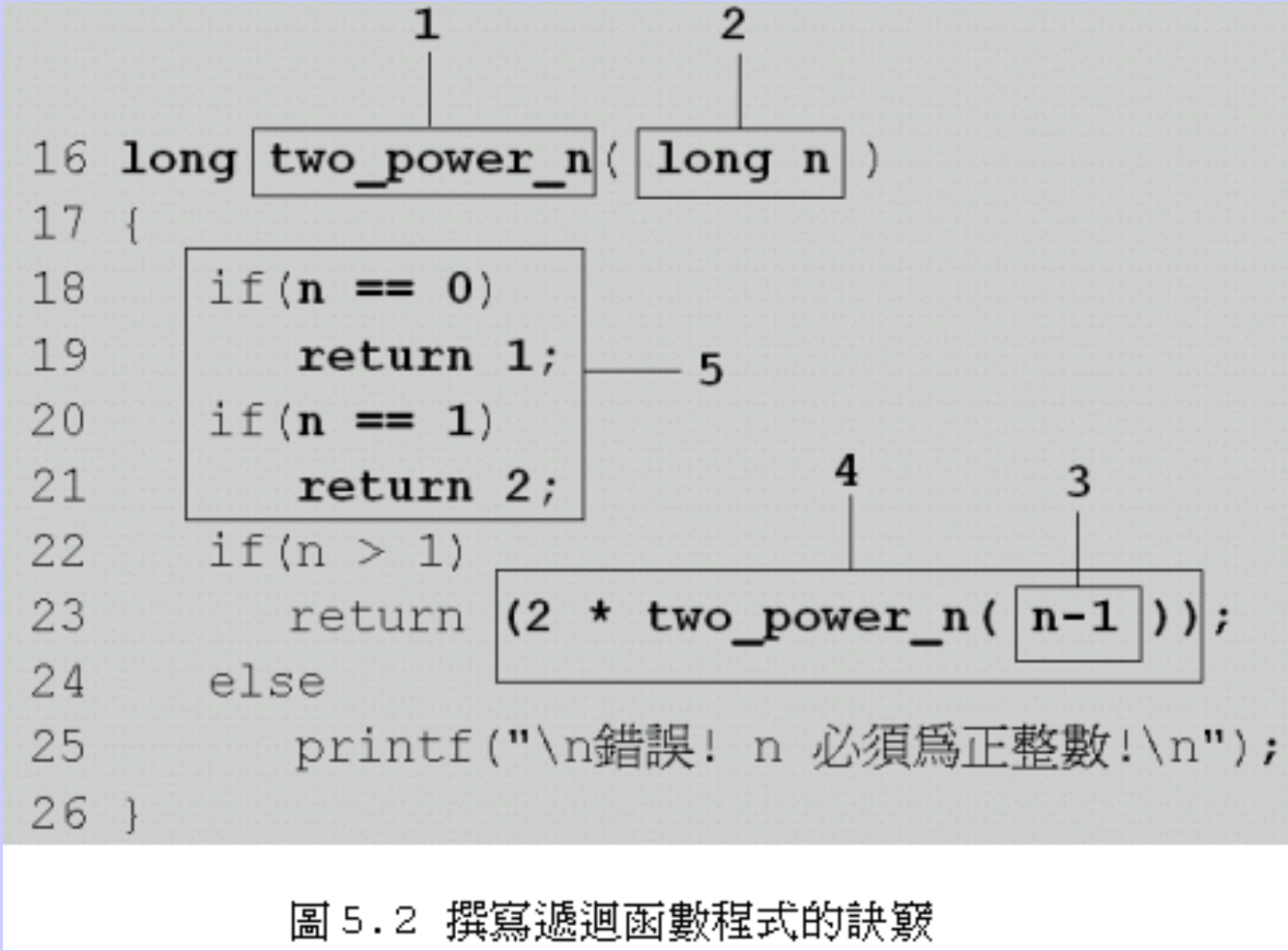


圖 5.2 撰寫遞迴函數程式的訣竅

5.4 遞迴的應用

- 5.4.1 計算 n 階乘 (n!)
- 5.4.2 列印費氏數列
- 5.4.3 列印河內塔的圓盤搬移過程
- 5.4.4 求兩個正整數的最大公因數
- 5.4.5 列出 n 筆不同資料所有可能的排列方式

5.4.1 計算 n 階乘 (n!)

$$
\begin{aligned}
n! &= n \times (n-1)! \\
&= n \times (n-1) \times (n-2)! \\
&= n \times (n-1) \times (n-2) \times (n-3)! \\
&= \ldots \\
&= n \times (n-1) \times (n-2) \times (n-3) \times \ldots \times (n-(n-2)) \times (n-(n-1)) \\
&= n \times (n-1) \times (n-2) \times (n-3) \times \ldots \times 2 \times 1 。
\end{aligned}
$$

5.4.1 計算 n 階乘 (n!)

表 5.1 撰寫計算 n 階乘遞迴函數程式須考慮的因素

撰寫遞迴函數須考慮的因素		解答
1	爲遞迴函數取一個響亮的名稱？	`factorial`
2	思考遞迴函數須要哪些引數？及每一個引數的資料型態？	`long n`
3	遞迴函數自己呼叫自己時要帶入的參數值爲何？	`n-1`
4	遞迴函數的回傳值爲何？如何在回傳時呼叫自己？	當參數值大於等於 1 時 `return (n * factorial(n-1))`
5	遞迴函數的終止條件爲何？終止遞迴時的回傳值爲何？	當參數值等於 1 時 `return 1`

5.4.1 計算 n 階乘 (n!)

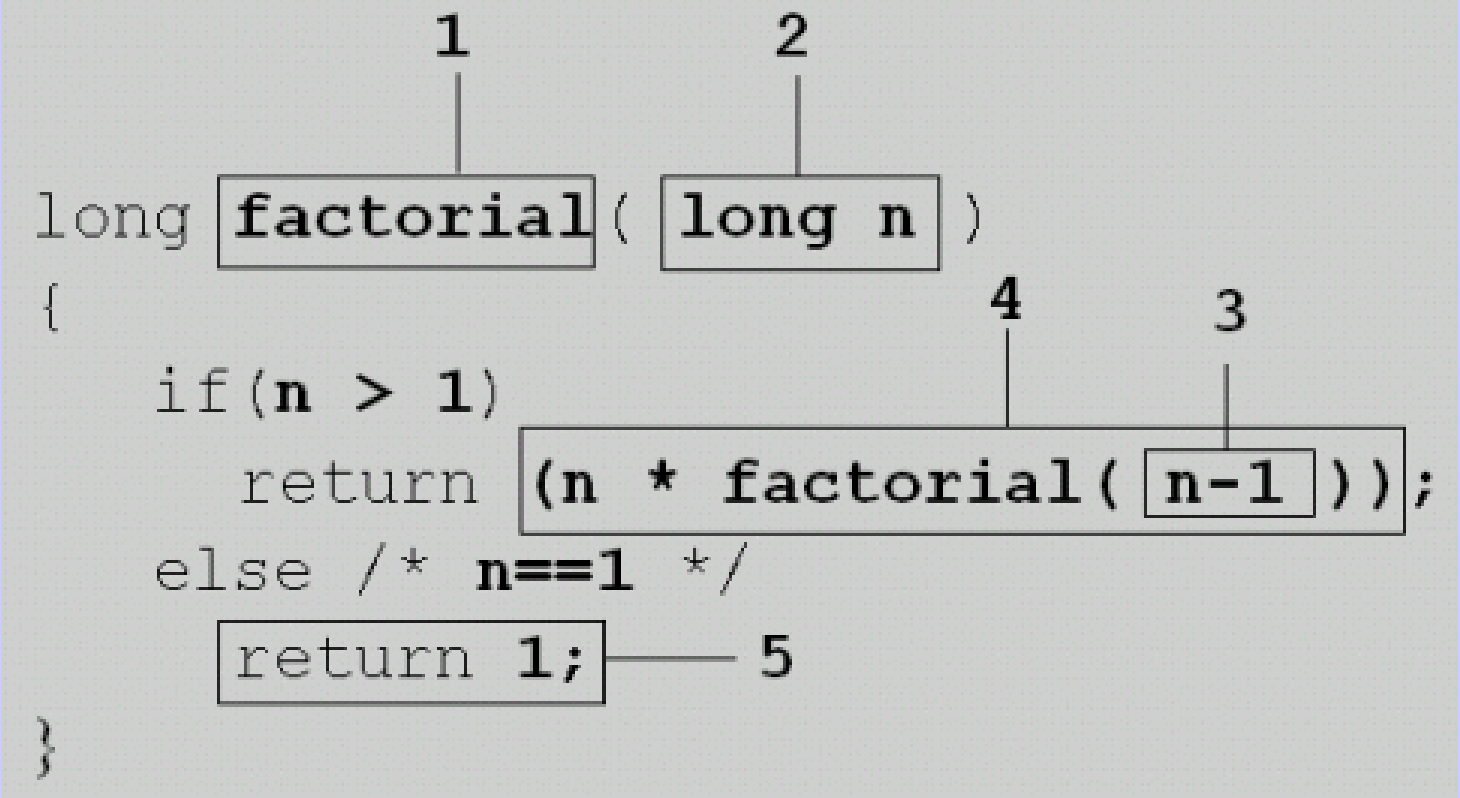


圖 5.3 計算出 n 階乘的遞迴函數程式

5.4.2 列印費氏數列

$$F(n)=\begin{cases} 0，當\ n = 0， \\ 1，當\ n = 1， \\ F_{n-1} + F_{n-2}，當\ n > 1。 \end{cases}$$

表 5.2 整理第 0 個至第 12 個費氏數之值

n	0	1	2	3	4	5	6	7	8	9	10	11	12
F(n)	0	1	1	2	3	5	8	13	21	34	55	89	144

5.4.2 列印費氏數列

```
13 /*-------------------------------------*/
14 /*   以遞迴方式列印出第 n 項費氏數          */
15 /*-------------------------------------*/
16 long fib(long n)
17 {
18    if(n == 0)
19       return 0;
20    if(n == 1)
21       return 1;
22    if(n > 1)
23       return (fib(n-1) + fib(n-2));
24    else
25       printf("\n 錯誤! n 必須爲大於 0 的整數!");
26 }
```

5.4.3 列印河內塔的圓盤搬移過程

【演算邏輯】將 n 個圓盤從 a 柱搬移至 c 柱之遞迴演算法

```
void hanoi(int disk_no,char source,char intermediate,char destination)
{
   static int step=0;

   if(disk_no > 1)
      hanoi(disk_no-1,source,destination,intermediate);
      printf("==>步驟(%d):將 %d 號圓盤從 %c 柱搬到 %c 柱。\n",++step,
            disk_no,source,destination);

   if(disk_no > 1)
      hanoi(disk_no-1,intermediate,source,destination);
}
```

5.4.4 求兩個正整數的最大公因數

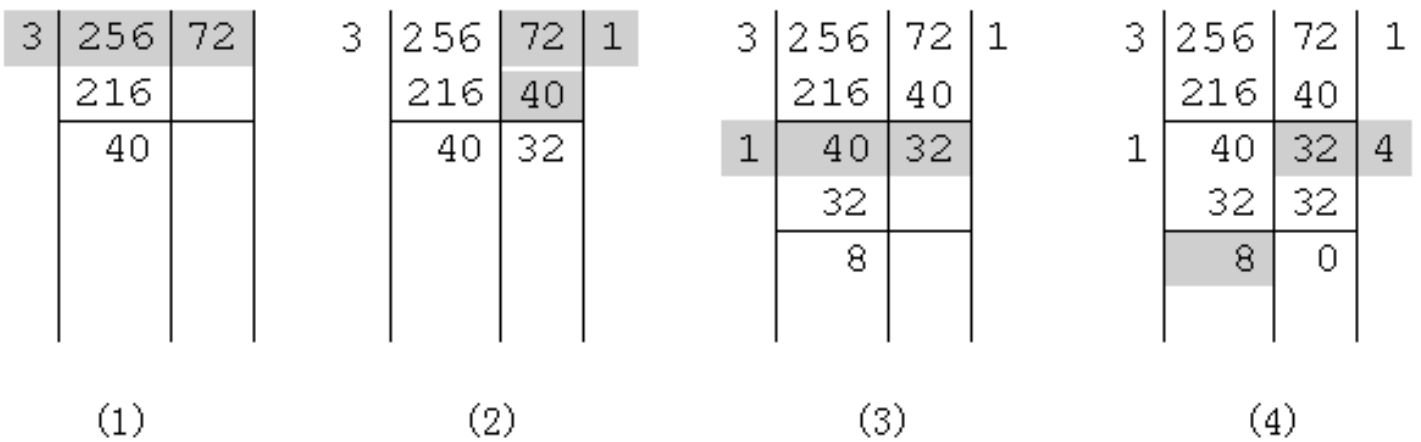


圖 5.4 利用輾轉相除法 gcd(256,72) 之過程

5.4.4 求兩個正整數的最大公因數

【演算邏輯】利用輾轉相除法求兩個正整數的最大公因數

```
步驟1：判斷兩個正整數中哪一個比較大。
       被除數(dividend) = 較大的正整數;
       除數(divisor) = 較小的正整數;
步驟2: 把被除數(dividend)除以除數(divisor)，得到商及餘數(reminder)。
步驟3: if(餘數(reminder) == 0)
          gcd = 除數(divisor);
          結束。
       else{
          被除數(dividend) = 除數(divisor);
          除數(divisor) = 餘數(reminder);
          跳到步驟2。
       }
```

5.4.5 列出 n 筆不同資料所有可能的排列方式

表 5.3 a、b、c 三筆資料的六種排列方式

排列方式	第 1 號位置	第 2 號位置	第 3 號位置
第 1 種排列方式	a	b	c
第 2 種排列方式	a	c	b
第 3 種排列方式	b	a	c
第 4 種排列方式	b	c	a
第 5 種排列方式	c	b	a
第 6 種排列方式	c	a	b